

MODULE POINTAGE

Dossier de Validation Finale & Tests Ultimes

Ce que «Terminé» veut dire — et comment on le prouve

Scope :	Check-in · Check-out · Today · Historique · Daily Summary · Quick Estimate · PDF
Bonus :	GPS · Horodatage serveur · RBAC · Isolation tenant · Concurrency · Performance
Structure :	8 chapitres · 85+ cas de test numérotés · Checklist de certification
Usage :	Lancer ces tests avant de déclarer le module «Production Ready»

85+

Cas de test

8

Scénarios majeurs

4

Rôles testés

100%

Coverage critique

TABLE DES MATIÈRES

01	Définition du «Terminé»	Critères de certification · Ce que ce document prouve · Comment l'utiliser
02	Périmètre complet du module	Tout ce qui doit exister · Ce qui est hors scope · Modèle de données
03	Scénario 1 — Check-in basique	Le cas d'usage fondateur · 12 cas de test
04	Scénario 2 — Check-out et calculs	Heures travaillées · Retard · Heures sup · 10 cas de test
05	Scénario 3 — Vue Today et temps réel	Manager multi-employés · Employé seul · 8 cas de test
06	Scénario 4 — Historique et filtres	Pagination · Filtres · Performance · 8 cas de test
07	Scénario 5 — Daily Summary	Calcul ce que je dois · Multi-type contrat · 8 cas de test
08	Scénario 6 — Quick Estimate & PDF	Simulation période · Génération PDF · 7 cas de test
09	Scénario 7 — Correction manuelle (manager)	Modifier un pointage · Audit trail · 8 cas de test
10	Scénario 8 — Sécurité & Isolation	RBAC · Tenant isolation · Concurrence · 12 cas de test
11	Tests de performance	Temps de réponse · Charge · Horodatage · 6 cas de test
12	Tests Flutter mobile	3 écrans · UX · Comportements limites · 8 cas de test
13	Checklist de certification finale	85 critères · Le document qui dit «c'est fini»

CH.01

DÉFINITION DU «TERMINÉ»

Ce document prouve que le module est production-ready

01

Le module Pointage est «Terminé» quand et seulement quand tous les tests de ce document passent avec le statut PASS. Pas 90%. Pas «presque tous». Tous. Les tests marqués CRITIQUE sont des bloquants absolus — un seul FAIL sur un test CRITIQUE = le module n'est pas livrable.

Les 4 niveaux de criticité

Niveau	Couleur	Signification	Comportement si FAIL
CRITIQUE	Rouge	Bug bloquant — l'utilisateur ne peut pas utiliser le module	Livraison impossible. Fix obligatoire avant tout autre test.
HAUTE	Ambre	Feature importante mal implémentée — impact fort sur l'expérience	Livraison impossible sauf exception documentée et approuvée.
MOYENNE	Vert	Feature secondaire imparfaite — impact mineur sur l'expérience	Livraison possible avec ticket Phase 2 créé.
BASSE	Gris	Polish, edge cases rares	Livraison possible. À corriger dans le sprint suivant.

Comment utiliser ce document

- 1. Préparer l'environnement** : base de données fraîche avec les seeders, au moins 1 company, 1 manager, 3 employés, 1 schedule configuré (8h-17h, 15min tolérance retard).
- 2. Exécuter les tests dans l'ordre** des chapitres. Certains tests dépendent du résultat des précédents.
- 3. Marquer chaque test** : PASS (tout est correct), FAIL (comportement différent du résultat attendu), BLOCK (impossible à tester à cause d'un autre bug), N/A (hors périmètre actuel).
- 4. Pour chaque FAIL** : noter le comportement réel observé, faire un screenshot, créer un ticket GitHub avec le numéro du test (ex: #PT-07).
- 5. La checklist finale** (Chapitre 13) est le document de livraison. Elle doit être signée (ou validée en commit) par le responsable du module.

Environnement de test requis

```
Environnement de test – setup initial

# Seeders à lancer avant les tests
php artisan migrate:fresh --seed
# Ce qui est créé :
# Company A : 'Tech Corp' – timezone: Africa/Algiers
# Company B : 'Autre Corp' – pour les tests d'isolation tenant
# Manager A : manager@techcorp.com / password123
# Employé 1 : alice@techcorp.com / password123
# Employé 2 : bob@techcorp.com / password123
# Employé 3 : carol@techcorp.com / password123 (archivé)
# Schedule : 'Standard' – 08:00-17:00 – lundi-vendredi – 15min tolérance
# Manager B (autre company) : manager@autrecorp.com / password123

# Variables d'environnement pour les tests
APP_TIMEZONE=Africa/Algiers
APP_ENV=testing
DB_CONNECTION=pgsql
# Heure serveur : doit être en UTC dans tous les logs
```

CH.02

PÉRIMÈTRE COMPLET DU MODULE

Tout ce qui doit exister — modèle de données — règles métier

02

Endpoints à valider

Méthode	Endpoint	Rôle	Description
POST	/api/v1/attendance/check-in	employee	Pointer l'arrivée (avec GPS optionnel)
POST	/api/v1/attendance/check-out	employee	Pointer le départ (avec GPS optionnel)
GET	/api/v1/attendance/today	manager + employee	État du jour — vue manager ou vue employé
GET	/api/v1/attendance	manager + employee	Historique paginé avec filtres date et employee_id
GET	/api/v1/estimation/daily-summary	manager	Daily Summary — ce que je dois aujourd'hui
GET	/api/v1/estimation/quick-estimate	manager	Quick Estimate — simulation sur période libre
GET	/api/v1/estimation/receipt-pdf	manager	Générer le reçu PDF d'une période

Règles métier à respecter — exhaustives

Règle	Description	Criticité
R-PT-01	L'horodatage du check-in est posé par le serveur en UTC. L'heure de l'appareil client est ignorée.	CRITIQUE
R-PT-02	Un employé ne peut pas faire deux check-in dans la même journée (même session_number). La constraint UNIQUE (employee_id, date, session_number) doit l'empêcher.	CRITIQUE
R-PT-03	Un check-out sans check-in préalable retourne une erreur MISSING_CHECK_IN (422).	CRITIQUE
R-PT-04	Un check-in quand un check-in est déjà ouvert retourne ALREADY_CHECKED_IN (422).	CRITIQUE
R-PT-05	Le statut est calculé au moment du check-in selon l'heure d'arrivée vs schedule.start_time + late_tolerance_minutes.	CRITIQUE
R-PT-06	hours_worked = (check_out - check_in - break_minutes) en heures décimales. Jamais négatif.	CRITIQUE
R-PT-07	Si hours_worked > overtime_threshold_daily → overtime_hours = hours_worked - threshold.	HAUTE
R-PT-08	late_minutes = max(0, (check_in - (schedule.start_time + tolerance))) en minutes.	HAUTE
R-PT-09	Un employé archivé (status=archived) ne peut pas se connecter ni pointer.	CRITIQUE

Règle	Description	Criticité
R-PT-10	Un manager ne peut voir les pointages que des employés de sa company. Jamais d'une autre company.	CRITIQUE
R-PT-11	Un employé ne peut voir que ses propres pointages via l'API.	CRITIQUE
R-PT-12	Le GPS est optionnel. Si fourni, gps_valid est calculé selon la proximité du site (gps_radius_m).	MOYENNE
R-PT-13	La correction manuelle (is_manual_edit=true) est réservée au manager. Elle trace edited_by et edit_reason.	HAUTE
R-PT-14	Le Daily Summary calcule uniquement les employés actifs avec un check-in ce jour.	HAUTE
R-PT-15	Le Quick Estimate accepte une période libre (date_from / date_to) et retourne le total estimé.	HAUTE
R-PT-16	Le PDF de reçu contient : nom employé, période, heures totales, taux, montant total, mention 'informatif'.	HAUTE
R-PT-17	Tous les horodatages retournés par l'API sont en ISO 8601 UTC.	CRITIQUE
R-PT-18	L'affichage heure dans l'app Flutter est en heure locale de la company (timezone).	HAUTE

CH.03

SCÉNARIO 1 — CHECK-IN

Le cas d'usage fondateur · La première action de chaque journée

03

1

Check-in basique (cas passants)

L'employé pointe son arrivée dans différentes conditions normales

7 tests

PT-01

Check-in nominal — avant l'heure

CRITIQUE

TODO

Employé `alice@techcorp.com`, schedule 08:00, heure serveur 07:45 UTC

ÉTAPES :

1. Se connecter avec `alice@techcorp.com`
2. POST `/api/v1/attendance/check-in` (body vide ou GPS optionnel)
3. Lire la réponse JSON
4. Vérifier la base de données

✓ RÉSULTAT ATTENDU :

HTTP 201. Réponse : `{id, employee_id, date=aujourd'hui, check_in=horodatage UTC, status='ontime', check_out=null, hours_worked=null}`. En base : ligne créée, check in en UTC, session number=1.

PT-02

Check-in avec retard toléré

CRITIQUE

TODO

Heure serveur 08:10 UTC, schedule 08:00, tolérance 15min

ÉTAPES :

1. POST `/api/v1/attendance/check-in` à 08:10 UTC
2. Vérifier le statut retourné

✓ RÉSULTAT ATTENDU :

status='ontime' car 10min < 15min de tolérance. late minutes=0.

PT-03

Check-in avec retard réel

CRITIQUE

TODO

Heure serveur 08:20 UTC, schedule 08:00, tolérance 15min

ÉTAPES :

1. POST `/api/v1/attendance/check-in` à 08:20 UTC
2. Vérifier status et late_minutes

✓ RÉSULTAT ATTENDU :

status='late'. late minutes=20. check in horodaté correctement en UTC.

PT-04

Check-in avec GPS valide

HAUTE

TODO

Employé sur le site principal (dans le rayon GPS)

ÉTAPES :

1. POST `/api/v1/attendance/check-in`
2. Body : `{ gps_lat: 36.752887, gps_lng: 3.042048 }` (dans le rayon du site)
3. Vérifier gps_valid en base

✓ RÉSULTAT ATTENDU :

HTTP 201. gps_lat et gps_lng stockés. gps_valid=true si dans le rayon, false si hors rayon. Pas d'erreur dans les deux cas (GPS est optionnel).

PT-05 Check-in sans schedule configuré

HAUTE

TODO

Employé bob@techcorp.com sans schedule assigné

ÉTAPES :

1. Retirer le schedule_id de bob en base
2. POST /api/v1/attendance/check-in avec bob
3. Vérifier le comportement

✓ RÉSULTAT ATTENDU :

HTTP 201. status='ontime' par défaut (pas de schedule = pas de calcul de retard). hours_worked restera null jusqu'au check-out.

PT-06 Check-in le week-end

MOYENNE

TODO

Heure serveur = samedi ou dimanche, schedule work_days=[1,2,3,4,5]

ÉTAPES :

1. Forcer la date serveur à un samedi (ou mocker)
2. POST /api/v1/attendance/check-in
3. Vérifier le statut

✓ RÉSULTAT ATTENDU :

HTTP 201. Le pointage est enregistré normalement. status='ontime' (le système ne bloque pas les pointages hors jours ouvrés — le manager peut avoir du week-end). Phase 2 : alerte visuelle sur le dashboard.

PT-07 Horodatage provient du serveur (pas du client)

CRITIQUE

TODO

Vérification que le client ne peut pas falsifier l'heure

ÉTAPES :

1. POST /api/v1/attendance/check-in avec header X-Client-Time: 2000-01-01T00:00:00Z
2. POST /api/v1/attendance/check-in avec body contenant check_in: '2000-01-01T00:00:00Z'
3. Vérifier la valeur stockée en base

✓ RÉSULTAT ATTENDU :

Dans les deux cas : le check_in en base = now() serveur UTC au moment de la requête. Le champ X-Client-Time et le champ check_in du body sont ignorés. Différence max tolérée entre stocké et now() : 2 secondes.

2

Check-in — cas d'erreur

Tentatives invalides qui doivent être rejetées proprement

5 tests

PT-08 Double check-in la même journée

CRITIQUE

TODO

Alice a déjà pointé son arrivée ce matin

ÉTAPES :

1. Alice fait un premier POST check-in → 201 OK
2. Alice refait un POST check-in dans la même journée
3. Vérifier le code HTTP et le message d'erreur

✓ RÉSULTAT ATTENDU :

HTTP 422. Body : { error_code: 'ALREADY_CHECKED_IN', message: 'Vous avez déjà pointé votre arrivée aujourd'hui.' }. Aucune nouvelle ligne créée en base.

PT-09 Check-in sans token d'authentification

CRITIQUE

TODO

Requête sans header Authorization

ÉTAPES :

1. POST /api/v1/attendance/check-in sans header Authorization
2. Vérifier la réponse

✓ RÉSULTAT ATTENDU :

HTTP 401. Body : { error_code: 'UNAUTHENTICATED' }. Pas d'effet en base.

PT-10 Check-in depuis le compte d'un manager

CRITIQUE

TODO

Un manager essaie de pointer comme un employé

ÉTAPES :

1. Se connecter avec manager@techcorp.com
2. POST /api/v1/attendance/check-in
3. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 403. Body : { error_code: 'FORBIDDEN' }. La policy interdit le check-in aux managers.**PT-11** Check-in d'un employé suspendu

CRITIQUE

TODO

Employé avec status=suspended essaie de pointer

ÉTAPES :

1. Suspendre carol en base (status=suspended)
2. Essayer de se connecter avec carol
3. Si connexion possible, essayer POST check-in

✓ RÉSULTAT ATTENDU :

Soit connexion impossible (403 à l'auth) soit check-in impossible (403 à l'action). Un employé suspendu ne peut pas pointer.**PT-12** Check-in d'un employé d'une autre company

CRITIQUE

TODO

Employé de Company B essaie de pointer dans Company A

ÉTAPES :

1. Se connecter avec un employé de Company B (token valide B)
2. POST /api/v1/attendance/check-in
3. Vérifier que le log créé appartient bien à Company B

✓ RÉSULTAT ATTENDU :

HTTP 201 mais le log est créé dans Company B — pas Company A. Le middleware tenant isole automatiquement. Aucune donnée de Company A n'est accessible ou modifiable.

CH.04

SCÉNARIO 2 — CHECK-OUT ET CALCULS

Heures travaillées · Retard · Heures supplémentaires

04

3

Check-out et calculs automatiques

Vérification de tous les calculs après le départ

10 tests

PT-13

Check-out nominal — journée complète

CRITIQUE

TODO

Alice a pointé à 08:00, check-out à 17:00, schedule 08:00-17:00, 60min pause

ÉTAPES :

1. Alice check-in à 08:00 (test PT-01 ou simulé)
2. Alice check-out à 17:00 : POST /api/v1/attendance/check-out
3. Vérifier hours_worked, overtime_hours, status

✓ RÉSULTAT ATTENDU :

HTTP 200. hours_worked = (17:00 - 08:00 - 60min) = 8.0h. overtime_hours = 0. status='ontime'. check_out horodaté en UTC.

PT-14

Calcul heures travaillées avec retard

CRITIQUE

TODO

Check-in 08:30, check-out 17:30, schedule 08:00-17:00, 60min pause

ÉTAPES :

1. Simuler check-in à 08:30 (status=late, late_minutes=30)
2. Check-out à 17:30
3. Vérifier hours_worked

✓ RÉSULTAT ATTENDU :

hours_worked = (17:30 - 08:30 - 60min) = 8.0h. Le retard ne réduit pas les heures travaillées — il est tracé séparément dans late_minutes. status='late' conservé.

PT-15

Calcul heures supplémentaires

HAUTE

TODO

Check-in 08:00, check-out 19:00, threshold=8h, pause=60min

ÉTAPES :

1. Simuler check-in 08:00, check-out 19:00
2. Vérifier overtime_hours

✓ RÉSULTAT ATTENDU :

hours_worked = (19:00 - 08:00 - 60min) = 10.0h. overtime_hours = 10.0 - 8.0 = 2.0h. Stockés séparément pour le calcul de paie.

PT-16

Calcul arrondi — fractions d'heures

HAUTE

TODO

Check-in 08:00, check-out 16:30, pause=60min

ÉTAPES :

1. Simuler ce check-in/check-out
2. Vérifier la précision de hours_worked

✓ RÉSULTAT ATTENDU :

hours_worked = (16:30 - 08:00 - 60min) = 7.5h. Stocké en DECIMAL(5,2). Valeur : 7.50. Pas d'arrondi à l'entier.

PT-17 Check-out sans check-in préalable

CRITIQUE

TODO

Tenter de checkout sans avoir checké in

ÉTAPES :

1. POST /api/v1/attendance/check-out sans check-in du jour 2. Vérifier le code HTTP et l'erreur

✓ RÉSULTAT ATTENDU :

HTTP 422. Body : { error_code: 'MISSING_CHECK_IN', message: 'Pointez d'abord votre arrivée avant de sortir.' }.**Aucune modification en base.****PT-18** Double check-out

CRITIQUE

TODO

Tenter de checkout après un checkout déjà fait

ÉTAPES :

1. Alice check-in puis check-out 2. Alice tente un second check-out 3. Vérifier la réponse

✓ RÉSULTAT ATTENDU :

HTTP 422. Error_code adapté (ex: ALREADY_CHECKED_OUT ou ALREADY_CHECKED_IN selon l'implémentation).**Pas de modification du check out existant.****PT-19** Check-out d'une journée qui n'est pas aujourd'hui

HAUTE

TODO

Tentative de checkout pour un check-in d'avant-hier

ÉTAPES :

1. Insérer manuellement un check-in sans check-out pour avant-hier 2. Alice tente de checkout aujourd'hui 3. Vérifier quel check-in est utilisé

✓ RÉSULTAT ATTENDU :

Le checkout s'applique au check-in ouvert le plus récent (qui devrait être celui d'aujourd'hui). Si le seul check-in ouvert est d'avant-hier, le comportement doit être documenté (MISSING_CHECK_IN ou checkout appliqué à l'ancien — à vérifier selon la règle métier choisie).**PT-20** Horodatage check-out provient du serveur

CRITIQUE

TODO

Vérification que le client ne peut pas falsifier l'heure de départ

ÉTAPES :

1. POST /api/v1/attendance/check-out avec body { check_out: '2000-01-01T00:00:00Z' } 2. Vérifier la valeur stockée

✓ RÉSULTAT ATTENDU :

check out en base = now() serveur UTC. Le champ du body est ignoré.**PT-21** hours_worked jamais négatif

HAUTE

TODO

Edge case : check-out avant check-in (bug de données ou fraude)

ÉTAPES :

1. Insérer manuellement un check-in dans le futur 2. Tenter un check-out immédiatement

✓ RÉSULTAT ATTENDU :

hours_worked ≥ 0. Le système ne stocke pas de valeur négative. Soit il refuse (422), soit il stocke 0. Comportement à documenter.

PT-22 Timezone — check-in à minuit heure locale

CRITIQUE

TODO

Employé en Algérie (UTC+1) qui pointe à 00:30 heure locale = 23:30 UTC veille

ÉTAPES :

1. Simuler un check-in à 23:30 UTC (00:30 Algérie) 2. Vérifier la date stockée dans le champ date

✓ RÉSULTAT ATTENDU :

La date stockée correspond à la date en heure locale de la company (Algérie = UTC+1). Donc date = le lendemain (jour où il est 00:30 localement), pas la veille.

CH.05

SCÉNARIO 3 — VUE TODAY

Vue temps réel du jour · Manager et employé · Multi-employés

05

4

GET /api/v1/attendance/today — tous les modes

Vue journalière — la requête la plus importante en production

8 tests

PT-23

Vue Today — manager sans filtre (tous les employés)

CRITIQUE

TODO

Manager demande l'état du jour sans spécifier d'employé

ÉTAPES :

1. GET /api/v1/attendance/today (manager token, pas de query param)
2. Vérifier la structure de la réponse

✓ RÉSULTAT ATTENDU :

HTTP 200. mode='collection'. Liste paginée de tous les employés actifs de la company avec pour chacun : employee_id, name, checked_in (bool), check_in_time (heure locale), check_out_time, hours_worked, status. Les employés non pointés apparaissent avec checked_in=false, status='absent'.

PT-24

Vue Today — manager pour un employé spécifique

HAUTE

TODO

Manager veut l'état du jour d'Alice

ÉTAPES :

1. GET /api/v1/attendance/today?employee_id=1
2. Vérifier mode et contenu

✓ RÉSULTAT ATTENDU :

HTTP 200. mode='single'. item contient les données du jour d'Alice uniquement.

PT-25

Vue Today — employé voit seulement ses données

CRITIQUE

TODO

Alice (employée) appelle today sans paramètre

ÉTAPES :

1. GET /api/v1/attendance/today (token Alice)
2. Vérifier qu'elle ne voit pas les autres

✓ RÉSULTAT ATTENDU :

HTTP 200. mode='single'. item = ses propres données du jour uniquement. Pas de liste d'autres employés.

PT-26

Vue Today — employé ne peut pas voir d'autre employé

CRITIQUE

TODO

Alice essaie de voir les données de Bob

ÉTAPES :

1. GET /api/v1/attendance/today?employee_id=2 (token Alice)
2. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 403. Une employée ne peut pas accéder aux données d'un autre employé.

PT-27 Vue Today — les heures sont en heure locale

CRITIQUE

TODO

Vérifier que `check_in_time` affiché est en heure locale, pas UTC

ÉTAPES :

1. Check-in à 07:00 UTC (= 08:00 heure Algérie)
2. GET `/api/v1/attendance/today`
3. Vérifier la valeur de `check_in_time` dans la réponse

✓ RÉSULTAT ATTENDU :

`check_in_time = '08:00'` (heure locale Algérie = UTC+1). Pas '07:00'. La conversion est faite côté serveur avec la `timezone` de la `companv`.

PT-28 Vue Today — pagination manager

HAUTE

TODO

Company avec 60 employés actifs

ÉTAPES :

1. GET `/api/v1/attendance/today` (manager, 60 employés)
2. Vérifier la pagination
3. GET `/api/v1/attendance/today?per_page=10&page;=2`

✓ RÉSULTAT ATTENDU :

Réponse paginée avec meta: `{current page, per page, total}`. Page 2 retourne les employés 11-20. total=60.

PT-29 Vue Today — employé archivé n'apparaît pas

HAUTE

TODO

Carol est archivée (`status=archived`)

ÉTAPES :

1. GET `/api/v1/attendance/today` (manager)
2. Vérifier que Carol n'apparaît pas dans la liste

✓ RÉSULTAT ATTENDU :

Carol n'apparaît pas dans la collection manager. Seuls les employés actifs (`status=active`) sont inclus.

PT-30 Vue Today — isolation tenant

CRITIQUE

TODO

Manager de Company A ne voit pas les employés de Company B

ÉTAPES :

1. GET `/api/v1/attendance/today` (manager Company A)
2. Vérifier que les employés de Company B sont absents de la réponse

✓ RÉSULTAT ATTENDU :

La réponse ne contient que les employés de Company A. Les employés de Company B sont invisibles.

CH.06

SCÉNARIO 4 — HISTORIQUE ET FILTRES

Pagination · Filtres date · Performance · Tri

06

5

GET /api/v1/attendance — historique

Accès à l'historique complet des pointages avec filtres

8 tests

PT-31

Historique employé — ses propres données

CRITIQUE

TODO

Alice consulte son propre historique

ÉTAPES :

1. GET /api/v1/attendance (token Alice)
2. Vérifier qu'elle voit uniquement ses logs
3. Vérifier l'ordre (date DESC)

✓ RÉSULTAT ATTENDU :

HTTP 200. data = liste de ses logs uniquement. Tri : date DESC, puis id DESC. meta avec pagination. Aucun log d'un autre employé n'apparaît.

PT-32

Historique manager — tous les employés

HAUTE

TODO

Manager consulte tout l'historique sans filtre

ÉTAPES :

1. GET /api/v1/attendance (token manager)
2. Vérifier que tous les employés de la company apparaissent

✓ RÉSULTAT ATTENDU :

HTTP 200. data = logs de tous les employés actifs de la company. Tri date DESC.

PT-33

Filtre date_from et date_to

HAUTE

TODO

Manager filtre l'historique du 1er au 15 du mois

ÉTAPES :

1. GET /api/v1/attendance?date_from=2026-04-01&date_to=2026-04-15
2. Vérifier que seuls les logs dans cet intervalle apparaissent

✓ RÉSULTAT ATTENDU :

Tous les logs retournés ont date >= 2026-04-01 AND date <= 2026-04-15. Aucun log hors de cet intervalle.

PT-34

Filtre employee_id par le manager

HAUTE

TODO

Manager filtre l'historique d'Alice uniquement

ÉTAPES :

1. GET /api/v1/attendance?employee_id=1 (token manager)
2. Vérifier que seuls les logs d'Alice apparaissent

✓ RÉSULTAT ATTENDU :

Tous les logs retournés ont employee_id = celui d'Alice. Tri date DESC.

PT-35 Employé ne peut pas filtrer par un autre employee_id

CRITIQUE

TODO

Alice essaie de voir l'historique de Bob

ÉTAPES :

1. GET /api/v1/attendance?employee_id=2 (token Alice)
2. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 403. Alice ne peut accéder qu'à ses propres logs.**PT-36** Pagination — per_page et page

HAUTE

TODO

Historique avec 30 logs, per_page=10

ÉTAPES :

1. Créer 30 logs en base
2. GET /api/v1/attendance?per_page=10
3. Vérifier meta et naviguer pages

✓ RÉSULTAT ATTENDU :

meta.total=30, meta.per_page=10, meta.current_page=1. 10 logs retournés. GET ?per_page=10&page;=3 retourne les 10 derniers logs.**PT-37** Contenu d'un log dans l'historique

HAUTE

TODO

Vérifier tous les champs retournés pour un log

ÉTAPES :

1. Créer un log complet (check-in + check-out)
2. GET /api/v1/attendance et lire le premier item

✓ RÉSULTAT ATTENDU :

Chaque log contient : id, employee_id, date (YYYY-MM-DD), check_in (ISO 8601 UTC), check_out (ISO 8601 UTC ou null), method, hours_worked (decimal ou null), overtime_hours, status, late_minutes. Pas de données sensibles (IBAN, national_id...).**PT-38** Filtre combiné date + employee_id

HAUTE

TODO

Manager filtre Alice + les 7 derniers jours

ÉTAPES :

1. GET /api/v1/attendance?employee_id=1&date;_from=2026-04-17&date;_to=2026-04-23
2. Vérifier la conjonction des filtres

✓ RÉSULTAT ATTENDU :

Uniquement les logs d'Alice dans la période. Les deux filtres s'appliquent en AND.

CH.07

SCÉNARIO 5 — DAILY SUMMARY

Calcul de ce que je dois · Multi-types de contrat · Validation

07

6

GET /api/v1/estimation/daily-summary — calculs financiers

La fonctionnalité core — ce que je dois à mes employés aujourd'hui

8 tests

Le Daily Summary est la feature qui répond à la question 'Combien je dois aujourd'hui ?'. C'est LA raison d'être de Leopardo RH selon le PILOTAGE.md. Ses calculs doivent être exacts à la décimale près.

PT-39

Daily Summary — employé au salaire fixe mensuel

CRITIQUE

TODO

Alice : salary_base=80000 DZD, salary_type=fixed, 22 jours ouvrés ce mois

ÉTAPES :

1. Alice check-in à 08:00, check-out à 17:00 (8h travaillées)
2. GET /api/v1/estimation/daily-summary (manager)
3. Vérifier le montant calculé pour Alice

✓ RÉSULTAT ATTENDU :

Montant Alice = $80000 / 22 \text{ jours} \approx 3636.36 \text{ DZD}$. Le calcul divise le salaire mensuel par le nombre de jours ouvrés du mois. Précision : 2 décimales.

PT-40

Daily Summary — employé horaire

CRITIQUE

TODO

Bob : salary_type=hourly, hourly_rate=500 DZD, 8h travaillées

ÉTAPES :

1. Bob check-in à 08:00, check-out à 17:00 (8h net)
2. GET /api/v1/estimation/daily-summary
3. Vérifier le montant pour Bob

✓ RÉSULTAT ATTENDU :

Montant Bob = $500 \times 8.0 = 4000 \text{ DZD}$. Calcul direct : $\text{hourly_rate} \times \text{hours_worked}$. Si Bob a 2h sup à taux 1.5 : overtime = $500 \times 1.5 \times 2 = 1500 \text{ DZD}$ supplémentaires (si règle heure sup implémentée).

PT-41

Daily Summary — employé journalier

HAUTE

TODO

Employé avec salary_type=daily, taux=2000 DZD/jour

ÉTAPES :

1. Créer un employé salary_type=daily, salary_base=2000
2. Cet employé pointe et checkout
3. Vérifier le calcul

✓ RÉSULTAT ATTENDU :

Montant = 2000 DZD (1 jour travaillé). Pas de calcul horaire. Si l'employé pointe plus de 8h, le montant reste 2000 DZD (journalier = 1 jour = taux journalier).

PT-42

Daily Summary — employé absent (pas de check-in)

HAUTE

TODO

Carol (active mais pas de pointage ce jour)

ÉTAPES :

1. GET /api/v1/estimation/daily-summary
2. Vérifier si Carol apparaît et avec quel montant

✓ RÉSULTAT ATTENDU :

Carol n'apparaît pas dans la liste ou apparaît avec hours_worked=0 et montant=0. Elle ne génère pas de coût ce jour.

PT-43 Daily Summary — plusieurs employés simultanément

CRITIQUE

TODO

3 employés avec des types de contrat différents ont tous pointé

ÉTAPES :

1. Alice (fixed), Bob (hourly), et un 3ème (daily) ont tous checké in et out
2. GET /api/v1/estimation/daily-summary
3. Vérifier le total général

✓ RÉSULTAT ATTENDU :

Réponse contient : liste des employés avec leurs montants individuels + total_day = somme de tous les montants. Le total doit être exact à la décimale.

PT-44 Daily Summary — accès réservé au manager

CRITIQUE

TODO

Un employé essaie d'accéder au Daily Summary

ÉTAPES :

1. GET /api/v1/estimation/daily-summary (token Alice employée)
2. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 403. Le Daily Summary ne doit pas être accessible aux employés (données financières des collègues).

PT-45 Daily Summary — isolation tenant

CRITIQUE

TODO

Le manager de Company A ne voit que ses employés

ÉTAPES :

1. Company A et Company B ont toutes deux des employés qui ont pointé
2. GET /api/v1/estimation/daily-summary (token manager Company A)
3. Vérifier l'absence des employés de Company B

✓ RÉSULTAT ATTENDU :

Uniquement les employés de Company A dans la réponse. Les données de Company B sont invisibles.

PT-46 Daily Summary — performance

HAUTE

TODO

Company avec 50 employés ayant tous pointé

ÉTAPES :

1. Créer 50 logs de pointage en base
2. Mesurer le temps de réponse de GET /api/v1/estimation/daily-summary

✓ RÉSULTAT ATTENDU :

Réponse en moins de 500ms. Vérifier qu'il n'y a pas de requête N+1 (les calculs sont agrégés en SQL, pas en boucle PHP).

CH.08

SCÉNARIO 6 — QUICK ESTIMATE & PDF

Simulation période libre · Génération du reçu informatif

08

7

Quick Estimate et PDF — cas passants et limites

Simulation financière sur période libre + génération du reçu

7 tests

PT-47

Quick Estimate — période du mois en cours

CRITIQUE

TODO

Manager simule ce qu'il doit à Alice du 1er au 15 avril

ÉTAPES :

1. GET /api/v1/estimation/quick-estimate?employee_id=1&date;_from=2026-04-01&date;_to=2026-04-15 2. Alice a travaillé 10 jours dans cet intervalle (salary_type=fixed, 80000 DZD/mois) 3. Vérifier le montant calculé

✓ RÉSULTAT ATTENDU :

Montant estimé $\approx 80000 \times (10/22) \approx 36363.63$ DZD. La réponse inclut : period, employee, total_hours, total_amount, currency, note='Estimation informative uniquement'.

PT-48

Quick Estimate — période chevauchant deux mois

HAUTE

TODO

Simulation du 20 mars au 10 avril (jours ouvrés à calculer sur 2 mois)

ÉTAPES :

1. GET /api/v1/estimation/quick-estimate?employee_id=1&date;_from=2026-03-20&date;_to=2026-04-10 2. Vérifier que le calcul est cohérent

✓ RÉSULTAT ATTENDU :

Le montant est calculé sur les jours ouvrés réels dans cet intervalle (pas juste les jours calendaires). Comportement documenté et cohérent.

PT-49

Quick Estimate — date_to antérieure à date_from

HAUTE

TODO

Paramètres incohérents

ÉTAPES :

1. GET /api/v1/estimation/quick-estimate?date_from=2026-04-15&date;_to=2026-04-01 2. Vérifier la réponse

✓ RÉSULTAT ATTENDU :

HTTP 422. Validation error : date_to doit être >= date_from.

PT-50

Quick Estimate — employé sans logs dans la période

HAUTE

TODO

Estimation sur une période où l'employé n'a pas pointé

ÉTAPES :

1. GET /api/v1/estimation/quick-estimate?employee_id=1&date;_from=2025-01-01&date;_to=2025-01-31 2. (Alice n'avait pas de logs en janvier 2025)

✓ RÉSULTAT ATTENDU :

Réponse valide avec total_hours=0 et total_amount=0 ou basé sur jours calendaires selon la règle métier choisie. Pas d'erreur 500.

PT-51 Génération du PDF reçu

HAUTE

TODO

*Manager génère le reçu PDF d'Alice pour la semaine***ÉTAPES :**

1. GET /api/v1/estimation/receipt-pdf?employee_id=1&date_from=2026-04-14&date_to=2026-04-20
2. Vérifier le Content-Type et le contenu du PDF

✓ RÉSULTAT ATTENDU :

HTTP 200. Content-Type: application/pdf. Le PDF contient : nom de l'entreprise, nom de l'employé, période, tableau des jours pointés, total heures, montant total estimé, mention légale 'Document informatif — non contractuel'. Lisible et correctement formaté.

PT-52 PDF — employé sans données dans la période

MOYENNE

TODO

*Générer un PDF pour une période sans pointages***ÉTAPES :**

1. GET /api/v1/estimation/receipt-pdf pour une période vide
2. Vérifier que le PDF est généré quand même

✓ RÉSULTAT ATTENDU :

HTTP 200. PDF généré avec total=0 et message 'Aucun pointage sur cette période'. Pas d'erreur 500.

PT-53 PDF — performance de génération

HAUTE

TODO

*Mesurer le temps de génération***ÉTAPES :**

1. GET /api/v1/estimation/receipt-pdf avec 1 mois de données
2. Mesurer le temps de réponse

✓ RÉSULTAT ATTENDU :

Réponse en moins de 2 secondes. DomPDF est synchrone — pas de queue nécessaire pour ce volume.

CH.09

SCÉNARIO 7 — CORRECTION MANUELLE (MANAGER)

Modifier un pointage · Audit trail · Traçabilité

09

8

Correction et audit des pointages

Le manager peut corriger un pointage oublié ou erroné

8 tests

La correction manuelle est une fonctionnalité critique pour la paie : si Alice a oublié de pointer sa sortie, le manager doit pouvoir la corriger. Chaque correction doit être tracée (qui, quand, pourquoi).

PT-54

Correction d'un check-in oublié

HAUTE

TODO

Bob n'a pas pointé son arrivée hier matin

ÉTAPES :

1. Manager PATCH /api/v1/attendance/{log_id} avec {check_in: '2026-04-22T07:00:00Z', edit_reason: 'Oubli pointage Bob'}
2. Vérifier les champs en base après modification

✓ RÉSULTAT ATTENDU :

HTTP 200. check_in mis à jour. is_manual_edit=true. edited_by=id du manager. edit_reason='Oubli pointage Bob'. Le status et hours_worked sont recalculés automatiquement.

PT-55

Correction d'un check-out manquant

HAUTE

TODO

Alice a pointé son arrivée mais pas sa sortie hier

ÉTAPES :

1. Manager PATCH /api/v1/attendance/{log_id} avec {check_out: '2026-04-22T17:00:00Z', edit_reason: 'Oubli checkout Alice'}
2. Vérifier hours_worked après correction

✓ RÉSULTAT ATTENDU :

HTTP 200. check_out renseigné. hours_worked recalculé automatiquement. overtime_hours recalculé. is_manual_edit=true.

PT-56

Un employé ne peut pas se corriger lui-même

CRITIQUE

TODO

Alice essaie de modifier son propre log

ÉTAPES :

1. Alice PATCH /api/v1/attendance/{log_id_alice}
2. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 403. Un employé ne peut pas modifier ses propres logs. Seul le manager peut corriger.

PT-57

Correction sans edit_reason

HAUTE

TODO

Manager tente de corriger sans expliquer pourquoi

ÉTAPES :

1. PATCH /api/v1/attendance/{log_id} sans edit_reason
2. Vérifier la validation

✓ RÉSULTAT ATTENDU :

HTTP 422 si edit_reason est obligatoire (recommandé). Ou HTTP 200 avec edit_reason=null si optionnel. Le comportement doit être documenté et cohérent.

PT-58 Audit trail — traçabilité complète

HAUTE

TODO

Vérifier que les corrections sont auditable

ÉTAPES :

1. Faire une correction sur un log 2. GET /api/v1/attendance/{log_id} ou GET /api/v1/attendance?employee_id=X 3. Vérifier la présence des champs d'audit dans la réponse

✓ RÉSULTAT ATTENDU :

La réponse de l'endpoint historique inclut : `is_manual_edit`, `edited_by` (nom ou id du manager), `edit_reason` pour les logs corrigés. Le manager peut voir qui a fait quelle correction.

PT-59 Correction d'un employé d'une autre company

CRITIQUE

TODO

Manager Company A essaie de modifier un log de Company B

ÉTAPES :

1. PATCH /api/v1/attendance/{log_id_company_B} (token manager Company A) 2. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 404 (le log de Company B est invisible depuis Company A). Isolation tenant garantie.

PT-60 Correction de pointage — recalcul automatique

HAUTE

TODO

Vérifier que tous les champs dérivés sont recalculés après correction

ÉTAPES :

1. Log existant : `check_in=08:30`, `check_out=null`, `status=late`, `late_minutes=30` 2. Correction : `check_in=08:05` (dans la tolérance) 3. Vérifier les champs recalculés

✓ RÉSULTAT ATTENDU :

Après correction : `status='ontime'`, `late_minutes=0`. Le recalcul est automatique à chaque modification de `check_in` ou `check_out`.

PT-61 Correction impossible sur un log ancien (règle Phase 2)

BASSE

TODO

Comportement si l'on tente de corriger un log de 3 mois

ÉTAPES :

1. PATCH /api/v1/attendance/{log_très_ancien} 2. Vérifier si une règle limite la correction dans le temps

✓ RÉSULTAT ATTENDU :

Phase 1 : la correction est permise sans limite de temps. La limite temporelle est une feature Phase 2 (ex: impossible de corriger un log > 90 jours). Documenter ce comportement actuel.

CH.10

SCÉNARIO 8 — SÉCURITÉ & ISOLATION

RBAC · Tenant isolation · Concurrency · Authentification

10

9

Sécurité — tests exhaustifs

Tous les vecteurs d'accès non autorisé doivent être couverts

12 tests

PT-62

Token expiré ou révoqué

CRITIQUE

TODO

Utiliser un token Sanctum périmé ou révoqué

ÉTAPES :

1. Révoquer le token d'Alice (POST /api/v1/auth/logout) 2. Réutiliser l'ancien token sur GET /api/v1/attendance/today 3. Vérifier le code HTTP

✓ RÉSULTAT ATTENDU :

HTTP 401. Bodv : {error: {code: 'UNAUTHENTICATED'}}. Le token révoqué n'est plus valide.

PT-63

Token d'un autre utilisateur de la même company

CRITIQUE

TODO

Utiliser le token de Bob pour faire une action au nom d'Alice

ÉTAPES :

1. Envoyer POST /api/v1/attendance/check-in avec le token de Bob 2. Vérifier à qui appartient le log créé

✓ RÉSULTAT ATTENDU :

Le log est créé pour Bob (l'utilisateur authentifié), pas pour Alice. Il est impossible de pointer 'en tant que quelqu'un d'autre' via l'API mobile.

PT-64

Injection SQL via les paramètres

CRITIQUE

TODO

Tester la résistance aux injections SQL

ÉTAPES :

1. GET /api/v1/attendance?employee_id=1' OR '1'='1 2. GET /api/v1/attendance?date_from=2026-04-01' DROP TABLE attendance_logs-- 3. Vérifier les réponses

✓ RÉSULTAT ATTENDU :

HTTP 422 (validation error) ou 400. Aucune exécution SQL malveillante. Laravel Eloquent + validation Request protège nativement contre les injections SQL.

PT-65

Double check-in concurrent (race condition)

CRITIQUE

TODO

Deux requêtes check-in simultanées depuis le même compte

ÉTAPES :

1. Envoyer deux requêtes POST check-in simultanées depuis Alice (même milliseconde ou thread) 2. Vérifier qu'un seul log est créé

✓ RÉSULTAT ATTENDU :

Un seul log créé. La contrainte UNIQUE (employee_id, date, session_number) en PostgreSQL garantit l'atomicité. La seconde requête reçoit 422 ALREADY_CHECKED_IN.

PT-66 Accès cross-company via ID direct

CRITIQUE

TODO

Manager Company A essaie d'accéder à un log par son ID connaissant l'ID numérique

ÉTAPES :

1. Manager A : GET /api/v1/attendance?employee_id={id_employé_company_B}
2. Vérifier la réponse

✓ RÉSULTAT ATTENDU :

HTTP 404 ou liste vide. Le Global Scope company_id filtre automatiquement toutes les requêtes. L'ID de Company B est invisible depuis Company A.

PT-67 Rate limiting — flood de check-in

HAUTE

TODO

Envoyer 100 requêtes check-in en 1 seconde depuis le même IP

ÉTAPES :

1. Script curl ou k6 : 100 POST check-in en 1s
2. Vérifier le comportement après la limite

✓ RÉSULTAT ATTENDU :

HTTP 429 Too Many Requests après la limite configurée (ex: 60/min). Le rate limiting Laravel protège l'API. Le message inclut Retry-After.

PT-68 Compte suspendu — toutes les actions

CRITIQUE

TODO

Vérifier que la suspension bloque toutes les actions

ÉTAPES :

1. Suspendre alice (status=suspended)
2. Tenter : login, check-in, GET today, GET history
3. Vérifier les codes HTTP

✓ RÉSULTAT ATTENDU :

Toutes les tentatives retournent 403 ACCOUNT_SUSPENDED. Un compte suspendu est bloqué à toutes les couches (middleware vérifie le statut).

PT-69 CORS — requêtes cross-origin

HAUTE

TODO

Vérifier que seules les origines autorisées peuvent appeler l'API

ÉTAPES :

1. Faire une requête depuis une origine non autorisée (curl -H 'Origin: https://evil.com')
2. Vérifier les headers CORS retournés

✓ RÉSULTAT ATTENDU :

L'API retourne les headers CORS appropriés (Access-Control-Allow-Origin limité aux domaines Leopardo RH). Les requêtes d'origines non autorisées sont rejetées ou ignorées par le navigateur.

PT-70 Données sensibles absentes des réponses

CRITIQUE

TODO

Vérifier qu'aucune donnée sensible ne fuite dans les réponses

ÉTAPES :

1. GET /api/v1/attendance/today — lire tous les champs retournés
2. GET /api/v1/attendance — lire tous les champs retournés
3. Chercher : IBAN, national_id, password_hash, salary_base dans les réponses

✓ RÉSULTAT ATTENDU :

Aucun des champs suivants n'apparaît dans les réponses : iban, national_id, personal_email, password (hash), bank_account, salary_base. Seuls les champs liés au pointage sont exposés.

PT-71 Headers de sécurité HTTP

HAUTE

TODO

Vérifier la présence des headers de sécurité essentiels

ÉTAPES :

1. Faire n'importe quelle requête API
2. Inspecter les headers de réponse HTTP

✓ RÉSULTAT ATTENDU :

Présence de : X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security (si HTTPS), Referrer-Policy. Absence de : X-Powered-By (expose PHP version). Server (expose Nginx version).

PT-73 Token admin ne donne pas accès aux données tenant

HAUTE

TODO

Si un super-admin existe, il ne doit pas accéder aux données des tenants sans y être autorisé

ÉTAPES :

1. Si un compte super_admin existe, tenter GET /api/v1/attendance depuis ce compte
2. Vérifier que le middleware tenant bloque l'accès ou que l'accès en lecture seule est documenté

✓ RÉSULTAT ATTENDU :

Le super_admin est dans la table super_admins (schéma public). Il n'a pas de token Sanctum tenant. Les endpoints /api/v1/* nécessitent un token tenant valide. Comportement documenté.

CH.11

TESTS DE PERFORMANCE

Temps de réponse · Charge · Horodatage précis

11

PT-74 Performance check-in sous charge normale

HAUTE

TODO

Temps de réponse d'un check-in avec une DB de taille normale

ÉTAPES :

1. Base avec 500 logs existants
2. Mesurer le temps de réponse de POST /api/v1/attendance/check-in
3. Répéter 10 fois et noter le p95

✓ RÉSULTAT ATTENDU :

p95 < 200ms. Le check-in est une opération simple (1 INSERT + 1 SELECT pour vérifier le doublon). Aucune N+1 query.

PT-75 Performance GET /today avec 50 employés

HAUTE

TODO

Temps de réponse de la vue Today avec une company de 50 personnes

ÉTAPES :

1. 50 employés actifs, tous avec un log du jour
2. GET /api/v1/attendance/today
3. Mesurer le temps de réponse p95

✓ RÉSULTAT ATTENDU :

p95 < 300ms. La requête utilise les index (date, employee_id). Pas de N+1 query (les logs sont chargés en une seule requête avec whereIn).

PT-76 Performance Daily Summary avec 30 employés

HAUTE

TODO

Temps de réponse du calcul de ce que je dois

ÉTAPES :

1. 30 employés avec logs du jour et différents types de contrat
2. GET /api/v1/estimation/daily-summary
3. Mesurer le temps de réponse p95

✓ RÉSULTAT ATTENDU :

p95 < 500ms. Les calculs sont accélérés côté SOL autant que possible. Pas de boucle PHP sur les employés un par un.

PT-77 Précision de l'horodatage serveur

CRITIQUE

TODO

Vérifier que l'horodatage serveur est précis à la seconde

ÉTAPES :

1. POST check-in et noter l'heure de l'envoi (now() local)
2. Comparer avec check_in retourné dans la réponse
3. Calculer la différence

✓ RÉSULTAT ATTENDU :

La différence entre l'heure de la requête et check_in stocké est < 2 secondes. Jamais 0 (pas hardcodé), jamais > 5s (pas de lag serveur excessif).

PT-78 Pas de N+1 queries

HAUTE

TODO

Vérifier l'absence de requêtes N+1 avec Laravel Debugbar ou logs

ÉTAPES :

1. Activer les query logs Laravel (DB::enableQueryLog())
2. GET /api/v1/attendance/today avec 10 employés
3. Compter le nombre de requêtes SQL exécutées

✓ RÉSULTAT ATTENDU :

Nombre de requêtes SQL = constant (ne grandit pas avec le nombre d'employés). Max acceptable : 5 requêtes totales.

Pas de boucle qui fait 1 requête par employé.

PT-79 Index PostgreSQL — EXPLAIN ANALYZE

HAUTE

TODO

Vérifier que les index sont utilisés sur les requêtes critiques

ÉTAPES :

1. EXPLAIN ANALYZE SELECT * FROM attendance_logs WHERE date = '2026-04-23' AND employee_id = 1
2. EXPLAIN ANALYZE SELECT * FROM attendance_logs WHERE date >= '2026-04-01' AND date <= '2026-04-30'
3. Vérifier qu'il n'y a pas de Seq Scan sur de grandes tables

✓ RÉSULTAT ATTENDU :

Les deux requêtes utilisent Index Scan (ou Bitmap Index Scan). Pas de Sequential Scan. Les index (date, status) et (employee_id, date DESC) sont utilisés.

CH.12

TESTS FLUTTER MOBILE

3 écrans · UX · Comportements aux limites · Stabilité

12

Ces tests sont manuels sur appareil réel ou simulateur. Tester sur : iPhone SE (375px), iPhone 14 Pro Max (430px), Android Pixel 6. Pour chaque test, noter le device utilisé.

PT-80

Écran Login — connexion réussie

CRITIQUE

TODO

Employé se connecte avec ses identifiants corrects

ÉTAPES :

1. Ouvrir l'app sur un téléphone propre (pas de session)
2. Saisir `alice@techcorp.com / password123`
3. Taper sur Connexion
4. Observer la navigation vers l'écran principal

✓ RÉSULTAT ATTENDU :

Feedback visuel pendant le chargement (indicateur ou bouton désactivé). Navigation vers AttendanceScreen en < 3 secondes. Token stocké de façon sécurisée (jamais en clair dans les logs).

PT-81

Écran Login — mauvais mot de passe

CRITIQUE

TODO

Saisir des identifiants incorrects

ÉTAPES :

1. Saisir `alice@techcorp.com / mauvaispassword`
2. Observer le message d'erreur

✓ RÉSULTAT ATTENDU :

Message d'erreur humain affiché : 'Email ou mot de passe incorrect.' PAS le code brut `INVALID_CREDENTIALS`. Pas de plantage. Le champ mot de passe reste saisissable pour retrv.

PT-82

AttendanceScreen — état initial (pas encore pointé)

CRITIQUE

TODO

Alice ouvre l'app sans avoir pointé ce matin

ÉTAPES :

1. Se connecter avec Alice
2. Observer l'AttendanceScreen à l'ouverture

✓ RÉSULTAT ATTENDU :

PulseButton en état 'arrivée' (vert, libellé 'Pointer l'arrivée'). Statut du jour = 'Pas encore pointé'. Heure actuelle affichée. MetricCards à 0 ou vides.

PT-83

Check-in depuis Flutter — succès

CRITIQUE

TODO

Alice tape le bouton de pointage

ÉTAPES :

1. Taper sur le PulseButton 'Pointer l'arrivée'
2. Observer le feedback et le changement d'état

✓ RÉSULTAT ATTENDU :

Feedback haptic + animation. Indicateur de chargement pendant l'appel API. Succès : le bouton bascule en 'Pointer le départ'. L'heure de check-in s'affiche. Toast ou confirmation visuelle. Durée de l'opération < 3 secondes sur 4G.

PT-84 AttendanceScreen — état après check-in

CRITIQUE

TODO

L'écran reflète correctement l'état après pointage

ÉTAPES :

1. Après un check-in réussi, observer l'AttendanceScreen

✓ RÉSULTAT ATTENDU :

Heure d'arrivée affichée en heure locale (pas UTC). Badge de statut (Présent ou En retard). PulseButton en mode check-out. Le LeopardoBadge a la bonne couleur (vert=présent, ambre=retard).

PT-85 HistoryScreen — affichage et navigation

HAUTE

TODO

Alice consulte son historique de pointages

ÉTAPES :

1. Naviguer vers l'HistoryScreen (onglet ou menu) 2. Observer la liste des pointages 3. Scroller

✓ RÉSULTAT ATTENDU :

Liste des 20 derniers pointages affichée. Chaque tile : date, heure arrivée, heure départ (ou '-'), badge statut, heures travaillées. Scroll fluide. Pull-to-refresh fonctionnel.

PT-86 Mode hors ligne — comportement gracieux

HAUTE

TODO

Couper le réseau et essayer de pointer

ÉTAPES :

1. Couper le WiFi et les données mobiles 2. Essayer de taper le bouton check-in

✓ RÉSULTAT ATTENDU :

Message d'erreur clair : 'Pas de connexion internet. Vérifiez votre réseau et réessayez.' PAS de plantage. PAS de spinner infini. L'app reste utilisable (peut consulter le cache de l'historique si implémenté).

PT-87 Light mode sur iPhone — lisibilité

CRITIQUE

TODO

Vérifier que l'app est lisible en mode clair (light mode)

ÉTAPES :

1. Passer l'iPhone en mode clair (Settings → Display) 2. Ouvrir l'app 3. Vérifier chaque écran : Login, AttendanceScreen, HistoryScreen

✓ RÉSULTAT ATTENDU :

Tous les textes sont lisibles (contraste > 4.5:1). Aucun texte invisible (blanc sur blanc ou noir sur noir). Les badges sont visibles. Le PulseButton est identifiable.

PT-88 Session persistante — pas de reconnexion chaque jour

HAUTE

TODO

Vérifier que la session dure 90 jours

ÉTAPES :

1. Se connecter avec Alice 2. Fermer complètement l'app 3. Rouvrir l'app le lendemain (ou simuler en changeant la date système) 4. Observer si la reconnexion est demandée

✓ RÉSULTAT ATTENDU :

L'app s'ouvre directement sur AttendanceScreen sans demander de reconnexion. Le token Sanctum est encore valide (configuré pour 90 jours).

CH.13

CHECKLIST DE CERTIFICATION FINALE

Le document qui dit officiellement que le module est terminé

13

Cette checklist est le document de livraison officiel du module Pointage. Elle doit être complétée et signée (ou validée par un commit tagué) avant que le module soit déclaré Production Ready. Tous les items CRITIQUE doivent être ✓ PASS.

Bloc 1 — Fonctionnalités core (13 items CRITIQUE)

**PT-01 à PT-07 : tous les cas de check-in passants sont PASS**

Horodatage serveur · statuts · GPS

**PT-08 à PT-12 : tous les cas d'erreur check-in retournent le bon code HTTP**

422 ALREADY_CHECKED_IN · 401 · 403 · isolation tenant

**PT-13 à PT-18 : check-out et calculs sont corrects**

hours_worked · overtime · late_minutes · erreurs propres

**PT-07 + PT-20 + PT-77 : l'horodatage est TOUJOURS posé par le serveur**

Jamais par le client — vérification en base

**PT-22 : timezone — la date stockée est en heure locale de la company**

Pas en UTC pour le champ date

**PT-23 + PT-25 : TODAY manager (collection) et employé (single) fonctionnent**

Modes corrects · champs corrects

**PT-26 + PT-30 : un employé ne voit pas les données d'un autre · isolation tenant**

403 et non 404 — test critique RBAC

**PT-27 : les heures affichées sont en heure locale (pas UTC)**

check_in_time = heure locale

**PT-31 + PT-35 : historique — employé voit seulement ses données**

403 si autre employee_id

**PT-39 + PT-40 : Daily Summary calcule correctement par type de contrat**

fixed / hourly / daily — précision 2 décimales

**PT-44 + PT-45 : Daily Summary réservé manager · isolation tenant**

403 employé · données company uniquement

**PT-47 + PT-49 : Quick Estimate fonctionne · validation date_from/date_to**

Calcul correct · 422 si dates incohérentes

**PT-51 : PDF généré correctement avec toutes les mentions requises**

Content-Type PDF · contenu correct · mention informative

Bloc 2 — RBAC et sécurité (10 items CRITIQUE)**PT-10 : un manager ne peut pas faire check-in**

403 FORBIDDEN

**PT-11 : un employé suspendu ne peut pas pointer**

403 à l'auth ou à l'action

**PT-56 : un employé ne peut pas se corriger lui-même**

403

**PT-62 : token révoqué retourne 401**

Plus de session valide après logout

**PT-63 : le token identifie l'acteur — impossible de pointer 'en tant que'**

Le log créé appartient à l'auteur du token

**PT-64 : injections SQL impossibles**

Laravel validation + Eloquent protège

**PT-65 : race condition check-in — contrainte UNIQUE fonctionne**

Un seul log créé même en concurrent

**PT-66 : cross-company par ID direct impossible**

404 ou liste vide — Global Scope

**PT-70 : aucune donnée sensible dans les réponses API**

IBAN · national_id · salary_base absents

**PT-73 : token super_admin sans accès tenant non documenté**

Comportement explicitement défini

Bloc 3 — Qualité et robustesse (15 items HAUTE)**PT-14 : retard tracé séparément — n'affecte pas hours_worked**

late_minutes ≠ déduction sur heures



PT-15 : heures sup calculées et stockées séparément

overtime_hours distinct de hours_worked



PT-21 : hours_worked jamais négatif

Edge case checkout avant checkin



PT-29 + PT-43 : employé archivé absent du today manager et du daily summary

status=archived exclu



PT-32 + PT-34 : filtres historique manager fonctionnent

employee_id · date_from · date_to



PT-36 : pagination correcte sur historique

meta.total · current_page · per_page



PT-54 + PT-55 : correction manager fonctionne

check_in et check_out modifiables



PT-58 : audit trail complet sur les corrections

is_manual_edit · edited_by · edit_reason



PT-59 : correction cross-company impossible

404 car log invisible



PT-60 : recalcul automatique après correction

status · late_minutes · hours_worked



PT-67 : rate limiting actif

429 après la limite



PT-74 : check-in p95 < 200ms

Performance nominale



PT-75 : today 50 employés p95 < 300ms

Performance avec charge



PT-78 : pas de N+1 queries

Max 5 requêtes SQL sur /today



PT-79 : index PostgreSQL utilisés

EXPLAIN ANALYZE sans Seq Scan

Bloc 4 — Flutter mobile (8 items)

**PT-80 + PT-81 : Login flutter — succès et erreur humaine**

Message lisible · pas de code brut

**PT-82 + PT-83 : AttendanceScreen — état initial et check-in**

UX correcte · feedback < 3s

**PT-84 : état après check-in correctement reflété**

Heure locale · badge statut · bouton inversé

**PT-85 : HistoryScreen affiche correctement la liste**

Tiles · scroll · pull-to-refresh

**PT-86 : comportement hors ligne gracieux**

Message clair · pas de plantage

**PT-87 : light mode lisible sur iPhone**

Contraste WCAG AA

**PT-88 : session persistante 90 jours**

Pas de reconnexion chaque jour

**Pas de crash non géré sur aucun des 3 écrans**

Test en conditions réelles : réseau lent, données manquantes

Bloc 5 — Documentation et livraison**La base de données ne contient aucune colonne de test ou de debug en production**

Seeders de test non lancés en prod

**Les logs Laravel ne contiennent aucune donnée sensible (IBAN, password, national_id)**

grep dans les logs avant mise en prod

**L'endpoint /api/health retourne 200 avec db=connected**

Health check opérationnel

**Les migrations sont toutes appliquées (php artisan migrate:status → tous 'Ran')**

Aucune migration pending

**Le CHANGELOG.md liste toutes les fonctionnalités du module Pointage**

Document vivant à jour

**L'ERD est à jour avec les tables attendance_logs, schedules, sites**

Documentation cohérente avec le code



Les tests Pest couvrent > 80% du code du module Pointage

Coverage report généré et archivé

CERTIFICATION FINALE

Je soussigné(e) certifie que tous les tests de criticité CRITIQUE et HAUTE ont été exécutés et sont marqués PASS. Le module Pointage est déclaré Production Ready.

Responsable technique

Date de certification

Version du code (git tag)

Ce document est la référence finale de validation du Module Pointage de Leopardo RH. Version 1.0 — Avril 2026. Il doit être archivé avec le tag de release correspondant dans le repository GitHub.